Computations for thin covers

Human Compression project

Private author edition

The first complete program is a required finite certificate identifying the order96 source presentation with the marked affine model. The remaining GAP and Python programs are bounded validations of written arguments, including the thin-base counterexample and four sample moduli. They are not premises of the uniform proof. The source refers to GAP and an Atlas but supplies no full executable listing; its historical computational cost is unknown. Optional validation is not compared asymmetrically with an unknown source cost. Drivers, integrity tools and scoped Lean are inventoried separately. No length factor is claimed here.

Required finite presentation certificate

```
# The single required finite presentation identification, extracted from the
# accepted Topic E universal row [4,3,3]/[3,4,3]. Optional finite validations
# are in separate complete programs, not premises of the uniform proof.
Need:=function(b,m) if not b then Error(m);fi;end;
Vects:=Tuples([0,1],4);
PermOf:=f->PermList(List(Vects,v->Position(Vects,f(v))));
r0:=PermOf(v->[(v[1]+1)mod 2,v[2],v[3],v[4]]);
r1:=PermOf(v->[v[1],(v[1]+v[2])mod 2,v[3],(v[3]+v[4])mod 2]);
r2:=PermOf(v->[(v[1]+v[2])mod 2,v[2],(v[3]+v[4])mod 2,v[4]]);
r3:=PermOf(v->[v[1],v[2],v[3],(v[4]+1)mod 2]);
rr:=[r0,r1,r2,r3];G:=Group(rr);Need(Size(G)=96,"explicit base order");
F:=FreeGroup(4);x:=GeneratorsOfGroup(F);
rels:=Concatenation(List(x,a->a^2),[(x[1]*x[2])^4,(x[2]*x[3])^3,
 (x[3]*x[4])^4,(x[1]*x[3])^2,(x[1]*x[4])^2,(x[2]*x[4])^2,
 (x[1]*x[2]*x[3])^3,(x[2]*x[3]*x[4])^3]);
U:=F/rels;Need(Size(U)=96,"exact retained universal presentation");
h:=GroupHomomorphismByImages(U,G,GeneratorsOfGroup(U),rr);
Need(h<>fail and IsBijective(h),"actual marked source identification");
Print("SOURCE_BASE96_MARKED_IDENTIFICATION_PASS\n");
```

Optional base and voltage checks

```
# Optional validation of written intersection and voltage arguments.
Read("base_certificate.g");
sets:=Combinations([1..4]);subs:=List(sets,s->Subgroup(G,rr{s}));
for a in [1..16] do for b in [1..16] do
 Need(Intersection(subs[a],subs[b])=subs[Position(sets,Intersection(sets[a],sets[b]))],"base all-subset
 IP");
od;od;
V:=Group(r0,r0^r1,r3,r3^r2);Need(Size(V)=16 and IsElementaryAbelian(V)and IsNormal(G,V),"actual translation
kernel");
Need(Size(Group(r1,r2))=6 and Size(Intersection(V,Group(r1,r2)))=1,"split S3 quotient");
Edge:=Group(r0,r2,r3);VE:=Group(Concatenation(GeneratorsOfGroup(V),GeneratorsOfGroup(Edge)));
Need(Size(Edge)=16 and Size(VE)=32 and Size(Intersection(Edge,V))=8,"opposite edge union");
Need(Group(r0,r3,r0^r1,r2)=VE,"source sign rules connect all32 flags");
H0:=Group(r1,r2,r3);vertices:=RightCosets(G,H0);
edge1:=Set(Edge);edge2:=Difference(Set(VE),edge1);
a:=Filtered([1..4],i->not IsEmpty(Intersection(Set(vertices[i]),edge1)));
b:=Filtered([1..4],i->not IsEmpty(Intersection(Set(vertices[i]),edge2)));
Need(Length(a)=2 and Length(b)=2 and IsEmpty(Intersection(a,b)),"opposite source edges");
z:=function(g,i)
 if i<>2 then return 0;elif g in V then return 1;
 elif g*r2 in V then return -1;else return 0;fi;
end;
for g in Elements(G) do
 for i in [0..3] do for j in [0..3] do if j>i+1 then
  a:=g;v:=0;for k in [i,j,i,j]do v:=v+z(a,k);a:=a*rr[k+1];od;
  Need(a=g and v=0,"distant square integer voltage");
 fi;od;od;
```

```
 # Every middle section projects bijectively to the alternating quotient cycle.
 a:=g;v:=0;for k in [1,2,1,2,1,2]do v:=v+z(a,k);a:=a*rr[k+1];od;
 Need(a=g and AbsInt(v)=1,"primitive section voltage");
od;
hshift:=function(g)if g in V or g*r1 in V then return 0;else return 1;fi;end;
for g in Elements(G) do for i in [0..3]do
 Need(z(r1*g,i)=-z(g,i)+hshift(g*rr[i+1])-hshift(g),"literal rho1 lift");
 Need(z(r0*g,i)=z(g,i)and z(r3*g,i)=z(g,i),"outer voltage lifts");
 Need(z(r2*g,i)=-z(g,i),"rho2 voltage inversion");
od;od;
Print("BASE_DIAGNOSTICS_ALL256_IP_INTEGER_VOLTAGES_PASS\n");
```

Optional cover and counterexample checks

```
#!/usr/bin/env python3
"""Finite diagnostics of written constructions; not a proof for all moduli."""
import argparse,itertools,json
from collections import Counter

def need(x,msg):
    if not x:raise RuntimeError(msg)
def components(points,step,colors):
    left=set(points);out=[]
    while left:
        seen={min(left)};todo=list(seen)
        for x in todo:
            for i in colors:
                y=step(x,i)
                if y not in seen:seen.add(y);todo.append(y)
        need(seen<=left,'component escaped specified subgraph')
        out.append(frozenset(seen));left-=seen
    return out

def inspect(points,step):
    rank=4;checks=0
    for x in points:
        ys=[step(x,i) for i in range(rank)]
        need(len(set(ys+[x]))==5,'simple proper coloring')
        for i in range(rank):
            need(step(ys[i],i)==x,'involution')
            for j in range(i+2,rank):
                need(step(ys[i],j)==step(ys[j],i),'distant commutation');checks+=1
    need(len(components(points,step,range(rank)))==1,'connected maniplex')
    faces=[components(points,step,[j for j in range(rank)if j!=i])for i in range(rank)]
    lookup=[{x:k for k,f in enumerate(fs)for x in f}for fs in faces]
    fibres=Counter(tuple(d[x]for d in lookup)for x in points)
    posfaces=[[frozenset(points)]]+faces+[[frozenset(points)]]
    diamonds=[]
    for i in range(rank):
        for a in posfaces[i]:
            for b in posfaces[i+2]:
                if a&b:diamonds.append(sum(bool(a&f)and bool(b&f)for f in faces[i]))
    # Independently require the induced incidence order to be transitive.
    for i,j,k in itertools.combinations(range(rank),3):
        for a in faces[i]:
            for b in faces[j]:
                if a&b:
                    for c in faces[k]:
                        need(not(b&c)or bool(a&c),'face order transitivity')
    return {'flags':len(points),'faces':list(map(len,faces)),
      'flag_fibres':dict(sorted(Counter(fibres.values()).items())),
      'diamonds':dict(sorted(Counter(diamonds).items())),
      'distant_checks':checks},faces

def bit_example():
    base=list(itertools.product(range(2),repeat=5))
    cover=list(itertools.product(range(2),repeat=6))
    def step(x,i):
        y=list(x);y[i]^=1
        if i==0:y[4]^=x[1]
        if i==2:y[4]^=x[3]
        if len(x)==6:
            if i==1:y[5]^=x[2]
            if i==2:y[5]^=x[3]
```

```
        return tuple(y)
    bm,bf=inspect(base,step);qm,qf=inspect(cover,step)
    need(bm['faces']==[2,2,2,2]and set(bm['diamonds'])=={2},'thin base')
    need(set(bm['flag_fibres'])=={2},'unfaithful base')
    need(qm['faces']==[2,4,4,2]and qm['diamonds']=={2:8,4:16},'nonthin cover')
    need(set(qm['flag_fibres'])=={1},'faithful cover')
    for x in cover:
        for i in range(4):need(step(x,i)[:5]==step(x[:5],i),'cover projection')
    lifts=[]
    for i in range(4):
        for f in bf[i]:
            pre=[x for x in cover if x[:5]in f]
            cs=components(pre,step,[j for j in range(4)if j!=i])
            need(len(cs)==(1 if i in(0,3)else 2),'literal L3.5 hypothesis')
            if i in(1,2):
                for c in cs:need(len(c)==len(f)and {x[:5]for x in c}==set(f),'isomorphic split face')
            lifts.append({'rank':i,'components':len(cs)})
    def a(x):return x[5]^x[4]^(x[0]&x[1])
    def b(x):return x[5]^(x[1]&x[2])
    for i,lab in [(0,lambda x:x[0]),(1,lambda x:(x[1],a(x))),
                  (2,lambda x:(x[2],b(x))),(3,lambda x:x[3])]:
        expected={frozenset(x for x in cover if lab(x)==v)for v in {lab(x)for x in cover}}
        need(expected==set(qf[i]),'exact face potential formula')
    witnesses=[(0,a0,0,0,d,0)for a0,d in itertools.product(range(2),repeat=2)]
    need(len({(x[1],a(x))for x in witnesses})==4,'four distinct intervening faces')
    need(all(x[0]==0 and x[2]==0 and b(x)==0 for x in witnesses),'actual common interval')
    return {'base':bm,'cover':qm,'all_eight_face_preimages':lifts,'four_flags':witnesses}

def B(v):return(v[0],v[0]^v[1],v[2],v[2]^v[3])
def C(v):return(v[0]^v[1],v[1],v[2]^v[3],v[3])
def act(k,t,v):
    if t:v=B(v)
    for _ in range(k%3):v=B(C(v))
    return v

def family(m):
    z=(0,0,0,0);points=[(v,k,t)for v in itertools.product(range(2),repeat=4)
                        for k in range(3*m)for t in range(2)]
    marks=[((1,0,0,0),0,0),(z,0,1),(z,3*m-1,1),((0,0,0,1),0,0)]
    def mul(x,y):
        v,k,t=x;w,l,u=y;w=act(k,t,w)
        return(tuple(a^b for a,b in zip(v,w)),(k+(-1)**t*l)%(3*m),t^u)
    def step(x,i):return mul(x,marks[i])
    report,faces=inspect(points,step)
    need(report['flags']==96*m and report['faces']==[4,6*m,6*m,4],'family order and faces')
    need(set(report['flag_fibres'])=={1}and set(report['diamonds'])=={2},'faithful thin polytope')
    identity=(z,0,0);subs=[]
    for mask in range(16):
        colors=[i for i in range(4)if mask>>i&1];seen={identity};todo=[identity]
        for x in todo:
            for i in colors:
                y=step(x,i)
                if y not in seen:seen.add(y);todo.append(y)
        subs.append(seen)
    for a,b in itertools.product(range(16),repeat=2):
        need(subs[a]&subs[b]==subs[a&b],'all-subset family IP')
    qp=[(x,e)for x in points for e in range(2)]
    def qstep(x,i):return(step(x[0],i),x[1]^1)
    qr,qf=inspect(qp,qstep)
    need(qr['faces']==[4,12*m,12*m,4]and set(qr['diamonds'])=={2},'double face counts/thinness')
    need(set(qr['flag_fibres'])=={1},'double faithful')
    end_a=next(f for f in qf[0]if(identity,0)in f)
    end_b=next(f for f in qf[3]if(identity,0)in f)
    sections=components(end_a&end_b,qstep,[1,2])
    need(sorted(map(len,sections))==[6*m,6*m],'disconnected middle section')
    return {'modulus':m,'base':report,'double':qr,'all_subset_IP':256,
            'middle_components':list(map(len,sections))}

def main():
    ap=argparse.ArgumentParser();ap.add_argument('--out',required=True);a=ap.parse_args()
    result={'status':'PASS','finite_only':True,'L3.5':bit_example(),
            'family_samples':[family(m)for m in [1,2,3,5]]}
    open(a.out,'w').write(json.dumps(result,indent=2)+'\n')
    print('COVER_DIAGNOSTICS_PASS')
if __name__=='__main__':main()
```